2D Cricket Web Application

CS-4032: Web Programming - Assignment #02

Student Name: Salman Ahmed
Roll Number: 22i-0743
Section: B

GitHub Repository

Repository URL: https://github.com/SalmanAh/Cricket-Web.git

Table of Contents

1. Project Overview
2. Screenshots
3. Game Logic and Working Mechanism
4. Mapping of Probabilities to Power Bar
5. Implementation of Animations
6. Technologies Used
7. Conclusion

---

1. Project Overview

This project is a single-player 2D cricket batting game developed using React, HTML, CSS, and JavaScript. The application simulates cricket gameplay using a probability-based power bar system where outcomes are determined by the position of a moving slider rather than pure randomness.

The game features two distinct batting styles (Aggressive and Defensive), each with different risk-reward profiles. Players must time their shots by clicking on a moving slider that travels across a color-coded power bar, with each segment representing different outcomes based on their probabilities.

---

2. Screenshots

Note: All screenshots include a sticky note displaying Name, Roll Number, and Section as required.

Screenshot 1: Aggressive Batting in Action

[INSERT SCREENSHOT HERE]

Description: This screenshot shows the Aggressive batting mode in action. The power bar displays a large red "Wicket" zone occupying 40% of the bar, indicating high risk. The purple "6 Runs" segment (15%) and orange "4 Runs" segment (10%) show the high-reward potential. The moving slider is visible, and the instruction "Click to play your shot!" guides the player.

Screenshot 2: Defensive Batting in Action

[INSERT SCREENSHOT HERE]

Description: This screenshot demonstrates the Defensive batting mode. The red "Wicket" zone is significantly smaller (20% of the bar), while the gray "0 Runs" (25%) and green "1 Run" (25%) segments are much larger, representing the safer, more conservative approach. The boundary segments (4 and 6) are minimal (3% and 2% respectively).

Screenshot 3: Power Bar Visible During Gameplay

[INSERT SCREENSHOT HERE]

Description: A detailed view of the probability-based power bar system. The bar is divided into color-coded segments: Red (Wicket), Gray (0), Green (1), Yellow (2), Orange (3, 4), and Purple (6). The black slider with a golden arrow continuously moves across the bar. Probability values are displayed at the bottom (0, 0.40, 0.50, 0.60, 0.70, 0.75, 0.85, 1.00), showing the cumulative probability ranges.

Screenshot 4: Scoreboard Updates Reflecting Game Progress

[INSERT SCREENSHOT HERE]

Description: This screenshot captures the game in progress after several balls have been played. The scoreboard at the top displays:

- **Runs:** Current score with wickets fallen (e.g., 18/1)
- **Overs:** Completed overs and balls (e.g., 1.3)
- **Balls Left:** Remaining deliveries (e.g., 9)
- **Wickets Left:** Available wickets (e.g., 1)

The cricket field shows the batsman in position, and the commentary box at the bottom displays context-aware messages about the previous delivery.

Screenshot 5: Game Over Screen

[INSERT SCREENSHOT HERE]

Description: The game over screen appears when either all 12 balls are completed or all 2 wickets are lost. It displays:

- "Game Over!" heading
- Final Score (e.g., "Final Score: 24/2")

- Overs Played (e.g., "Overs: 2.0")
- "Restart Game" button to reset and play again

The popup appears centered over the cricket field with a smooth slide-in animation.

---

3. Game Logic and Working Mechanism

3.1 Core Game Flow

The game follows a structured sequence of states and transitions:

State Management

The application uses React hooks to manage the following states:

- `runs`: Total runs scored by the player
- `wickets`: Number of wickets fallen
- `ballsPlayed`: Count of balls bowled
- `battingStyle`: Selected style ('aggressive' or 'defensive')
- `gameState`: Current game phase ('ready', 'bowling', 'batting', 'shotPlayed', 'result', 'gameOver')
- `lastOutcome`: Result of the most recent delivery
- `showPowerBar`: Controls power bar visibility

Game Sequence

Step 1: Initialization

- Game starts with 0 runs, 0 wickets, 0 balls played
- Player has 12 balls (2 overs) and 2 wickets available
- Game state is set to 'ready'

Step 2: Batting Style Selection

- Player chooses between Aggressive or Defensive batting
- Each style has different probability distributions
- Selection affects the power bar segments

Step 3: Ball Delivery

- Player clicks "Play Ball" button
- Game state changes to 'bowling'
- Bowling animation plays (ball travels from right to batsman)
- Duration: 1.5 seconds

Step 4: Power Bar Interaction

- After bowling animation completes, game state changes to 'batting'
- Power bar appears with moving slider
- Slider continuously moves from left (0) to right (1)
- Slider completes one cycle in 2 seconds
- Player must click on the power bar to stop the slider

Step 5: Outcome Determination

- When player clicks, slider position is captured (value between 0 and 1)
- System finds which probability segment contains the slider position
- Outcome is determined based on the segment
- Game state changes to 'shotPlayed'

Step 6: Shot Animation

- Batting animation plays (bat swings)
- Ball animation plays:
    - If outcome is a score (0-6): Ball flies to the right
    - If outcome is wicket: Ball moves toward stumps, stumps break
- Duration: 0.6-0.8 seconds

Step 7: Result Display

- Game state changes to 'result'
- Outcome is displayed on screen (e.g., "FOUR!", "OUT!", "2 Runs")
- Commentary appears with context-aware message
- Score is updated (runs added or wicket incremented)
- Ball count increases by 1

Step 8: Next Ball or Game Over

- After 2 seconds, result display disappears
- System checks game end conditions:
  - If ballsPlayed >= 12 OR wickets >= 2: Game Over
  - Otherwise: Game state returns to 'ready' for next ball

Game End Conditions

The game ends when either condition is met:

1. All 12 balls (2 overs) have been bowled
2. All 2 wickets have fallen

When game ends:

- Game state changes to 'gameOver'
- Final score is displayed
- "Restart Game" button appears
- Clicking restart resets all state variables to initial values

3.2 Component Architecture

App.js (Main Controller)

- Manages all game state
- Handles game flow logic
- Coordinates between components
- Implements game end detection using useEffect hook

CricketField.js

- Renders the cricket ground visual
- Displays pitch, batsman, ball, and stumps
- Handles all animations (bowling, batting, ball hit, wicket)
- Shows result display overlay

Scoreboard.js

- Displays current runs and wickets
- Shows overs and balls played
- Calculates and displays remaining balls and wickets

- Updates in real-time after each delivery

PowerBar.js

- Implements probability-based power bar
- Calculates cumulative probability ranges
- Animates slider movement using requestAnimationFrame
- Determines outcome based on slider position
- Renders color-coded segments proportional to probabilities

BattingStyleSelector.js

- Provides UI for selecting batting style
- Displays style characteristics (risk/reward)
- Contains "Play Ball" button to initiate delivery
- Shows active style with visual feedback

Commentary.js

- Displays dynamic commentary after each ball
- Randomly selects from multiple pre-defined statements
- Different messages for each outcome type
- Adds engagement and realism to gameplay

---

4. Mapping of Probabilities to Power Bar

4.1 Probability Distributions

The game uses two distinct probability distributions for the two batting styles:

Aggressive Batting Style

Outcome	Probability	Percentage	Power Bar Range
Wicket	0.40	40%	0.00 - 0.40

Outcome	Probability	Percentage	Power Bar Range
0 Runs	0.10	10%	0.40 - 0.50
1 Run	0.10	10%	0.50 - 0.60
2 Runs	0.10	10%	0.60 - 0.70
3 Runs	0.05	5%	0.70 - 0.75
4 Runs	0.10	10%	0.75 - 0.85
6 Runs	0.15	15%	0.85 - 1.00
Total	**1.00**	**100%**	**-**

Characteristics:

- High risk: 40% chance of getting out
- High reward: 25% chance of boundaries (4 or 6)
- Suitable for aggressive players willing to take risks

Defensive Batting Style

Outcome	Probability	Percentage	Power Bar Range
Wicket	0.20	20%	0.00 - 0.20
0 Runs	0.25	25%	0.20 - 0.45
1 Run	0.25	25%	0.45 - 0.70
2 Runs	0.15	15%	0.70 - 0.85
3 Runs	0.10	10%	0.85 - 0.95
4 Runs	0.03	3%	0.95 - 0.98
6 Runs	0.02	2%	0.98 - 1.00
Total	**1.00**	**100%**	**-**

Characteristics:

- Low risk: 20% chance of getting out
- Low reward: 5% chance of boundaries (4 or 6)
- Suitable for defensive players prioritizing survival

4.2 How Probabilities Map to Power Bar

Visual Representation

Each segment of the power bar is sized proportionally to its probability:

Segment Width = Probability × 100%

For example, in Aggressive mode:

- Wicket segment width = 0.40 × 100% = 40% of total bar width
- 6 Runs segment width = 0.15 × 100% = 15% of total bar width
- 3 Runs segment width = 0.05 × 100% = 5% of total bar width

Color Coding

Each outcome is assigned a distinct color for easy identification:

- **Red:** Wicket (danger zone)
- **Gray:** 0 Runs (no score)
- **Green:** 1 Run (safe single)
- **Yellow:** 2 Runs
- **Orange:** 3 Runs and 4 Runs
- **Purple:** 6 Runs (maximum reward)

Cumulative Range Calculation

The implementation uses cumulative probability ranges:

```
const ranges = [];
let cumulative = 0;

['wicket', '0', '1', '2', '3', '4', '6'].forEach(outcome => {
  const prob = currentProbs[outcome];
  ranges.push({
    outcome: outcome,
    start: cumulative,
    end: cumulative + prob,
    probability: prob
  });
  cumulative += prob;
});
```

This creates non-overlapping ranges that cover the entire 0-1 spectrum.

4.3 Outcome Determination Algorithm

When the player clicks on the power bar:

1. **Capture Slider Position:** The current slider position (0-1) is recorded
2. **Find Matching Range:** The system searches through the probability ranges
3. **Determine Outcome:** The outcome corresponding to the range containing the slider position is selected

```
const outcome = ranges.find(range =>
  sliderPosition >= range.start && sliderPosition < range.end
);
```

Example Scenarios

Scenario 1: Aggressive Batting

- Slider stops at position: 0.32
- Check ranges: 0.32 falls within [0.00 - 0.40]
- **Outcome: Wicket**

Scenario 2: Aggressive Batting

- Slider stops at position: 0.88
- Check ranges: 0.88 falls within [0.85 - 1.00]
- **Outcome: 6 Runs**

Scenario 3: Defensive Batting

- Slider stops at position: 0.55
- Check ranges: 0.55 falls within [0.45 - 0.70]
- **Outcome: 1 Run**

Scenario 4: Defensive Batting

- Slider stops at position: 0.15

- Check ranges: 0.15 falls within [0.00 - 0.20]
- **Outcome: Wicket**

This system ensures that outcomes are NOT random but are deterministic based on player timing and skill in stopping the slider at the desired position.

---

5. Implementation of Animations

5.1 Bowling Animation

Purpose

Simulates the ball being bowled from the bowler to the batsman.

Implementation

```
@keyframes bowlBall {
  0% {
    right: 100%;
    top: 30%;
    opacity: 1;
  }
  100% {
    right: 150px;
    top: 50%;
    opacity: 1;
  }
}

.bowling-animation {
  animation: bowlBall 1.5s ease-in forwards;
  opacity: 1;
}
```

How It Works

- Ball starts from the right edge of the screen (right: 100%)
- Travels to the batsman position (right: 150px)
- Vertical position changes from 30% to 50% (slight arc)
- Duration: 1.5 seconds with ease-in timing

- Triggered when game state changes to 'bowling'
- Animation fills forwards to maintain final position

5.2 Batting Animation

Purpose

Simulates the batsman playing a shot by swinging the bat.

Implementation

```
@keyframes batSwing {
  0% {
    transform: rotate(-45deg);
  }
  50% {
    transform: rotate(20deg);
  }
  100% {
    transform: rotate(-45deg);
  }
}

.batting-animation .bat {
  animation: batSwing 0.5s ease-in-out;
}
```

How It Works

- Bat starts at rest position (-45 degrees)
- Swings forward to 20 degrees (shot played)
- Returns to rest position
- Duration: 0.5 seconds with ease-in-out timing
- Triggered when player clicks on power bar (game state: 'shotPlayed')
- Transform origin is set to left center for realistic swing

5.3 Ball Hit Animation (For Scores)

Purpose

Shows the ball flying away after being hit by the batsman (when outcome is 0, 1, 2, 3, 4, or 6 runs).

Implementation

```
@keyframes ballHit {
  0% {
    right: 150px;
    top: 50%;
    opacity: 1;
    transform: translateY(-50%) scale(1);
  }
  30% {
    right: 100px;
    top: 40%;
    transform: translateY(-50%) scale(1.3);
  }
  100% {
    right: -100px;
    top: 10%;
    opacity: 0;
    transform: translateY(-50%) scale(0.8);
  }
}

.ball-hit {
  animation: ballHit 0.8s ease-out forwards;
  opacity: 1;
}
```

How It Works

- Ball starts at batsman position (right: 150px)
- Briefly enlarges (scale 1.3) to show impact
- Flies toward the right side of the screen
- Moves upward (top: 10%) simulating a lofted shot
- Fades out (opacity: 0) as it exits
- Duration: 0.8 seconds
- Triggered when outcome is NOT a wicket

5.4 Ball Hit Wicket Animation (For Wickets)

Purpose

Shows the ball hitting the stumps when the batsman gets out.

Implementation

```
@keyframes ballHitWicket {
  0% {
    right: 150px;
    top: 50%;
    opacity: 1;
  }
  50% {
    right: 200px;
    top: 50%;
    opacity: 1;
  }
  100% {
    right: 250px;
    top: 50%;
    opacity: 0;
  }
}

.ball-hit-wicket {
  animation: ballHitWicket 0.6s ease-out forwards;
  opacity: 1;
}
```

How It Works

- Ball moves from batsman position toward the stumps (left side)
- Travels horizontally at the same height
- Fades out near the stumps
- Duration: 0.6 seconds
- Triggered when outcome is 'wicket'

5.5 Stumps Breaking Animation

Purpose

Visual effect showing stumps falling when batsman is out.

Implementation

```
@keyframes stumpFall {
  0% {
    transform: rotate(0deg) translateY(0);
    opacity: 1;
  }
  100% {
    transform: rotate(45deg) translateY(20px);
    opacity: 0.3;
  }
}

.stumps-broken .stump {
  animation: stumpFall 0.5s ease-out forwards;
}
```

How It Works

- Each stump rotates 45 degrees
- Stumps fall downward (translateY: 20px)
- Opacity reduces to 0.3 to show broken state
- Each stump has slight delay for staggered effect
- Triggered when outcome is 'wicket'

5.6 Wicket Animation (Batsman Falls)

Purpose

Shows the batsman falling or reacting to being dismissed.

Implementation

```
@keyframes wicketFall {
  0% {
    transform: translateY(50%) rotate(0deg);
```

```
    opacity: 1;
  }
  100% {
    transform: translateY(50%) rotate(90deg) translateX(30px);
    opacity: 0.5;
  }
}

.wicket-animation {
  animation: wicketFall 0.5s ease-out;
}
```

How It Works

- Batsman rotates 90 degrees (falling motion)
- Moves sideways (translateX: 30px)
- Opacity reduces to show defeat
- Duration: 0.5 seconds
- Triggered when outcome is 'wicket'

5.7 Result Display Animation

Purpose

Shows the outcome text with an eye-catching pop effect.

Implementation

```
@keyframes resultPop {
  0% {
    transform: translate(-50%, -50%) scale(0);
    opacity: 0;
  }
  50% {
    transform: translate(-50%, -50%) scale(1.2);
  }
  100% {
    transform: translate(-50%, -50%) scale(1);
    opacity: 1;
  }
}

.result-display {
  animation: resultPop 0.5s ease-out;
```

```
}
```

How It Works

- Text starts at scale 0 (invisible)
- Quickly scales to 1.2 (overshoots)
- Settles at scale 1 (final size)
- Creates a "pop" or "bounce" effect
- Color-coded: Red (wicket), Gold (boundaries), Green (runs)

5.8 Slider Movement Animation

Purpose

Continuously moves the slider across the power bar for player interaction.

Implementation

```
useEffect(() => {
  if (!isMoving) return;

  const animate = () => {
    const elapsed = Date.now() - startTimeRef.current;
    const position = (elapsed % 2000) / 2000;
    setSliderPosition(position);
    animationRef.current = requestAnimationFrame(animate);
  };

  animationRef.current = requestAnimationFrame(animate);

  return () => {
    if (animationRef.current) {
      cancelAnimationFrame(animationRef.current);
    }
  };
}, [isMoving]);
```

How It Works

- Uses requestAnimationFrame for smooth 60fps animation

- Calculates position based on elapsed time
- Position value cycles between 0 and 1 every 2 seconds
- Stops when player clicks (isMoving set to false)
- Cleanup function cancels animation on component unmount

5.9 Commentary Slide-Up Animation

Purpose

Smoothly displays commentary text after each delivery.

Implementation

```
@keyframes slideUp {
  from {
    opacity: 0;
    transform: translateY(20px);
  }
  to {
    opacity: 1;
    transform: translateY(0);
  }
}

.commentary {
  animation: slideUp 0.5s ease-out;
}
```

How It Works

- Commentary box starts below its final position
- Slides upward while fading in
- Duration: 0.5 seconds
- Creates smooth entrance effect

---

6. Technologies Used

Frontend Framework

- **React 18:** Component-based UI library
- **React Hooks:** useState, useEffect, useRef for state management

Core Web Technologies

- **HTML5:** Semantic markup structure
- **CSS3:** Styling, animations, gradients, flexbox
- **JavaScript ES6+:** Modern JavaScript features (arrow functions, destructuring, template literals)

Rendering Approach

- **DOM-based rendering:** Using React components and CSS
- No Canvas API used (pure DOM manipulation)

Animation Techniques

- **CSS Keyframe Animations:** For bowling, batting, ball, and result animations
- **RequestAnimationFrame:** For smooth slider movement
- **CSS Transitions:** For button hover effects and smooth state changes

Version Control

- **Git:** Source code version control
- **GitHub:** Remote repository hosting

Development Tools

- **Create React App:** Project scaffolding and build configuration
- **npm:** Package management
- **ES6 Modules:** Code organization and imports

---

7. Conclusion

This 2D Cricket Web Application successfully implements all required features as specified in the assignment. The game provides an engaging cricket batting experience with realistic mechanics driven by probability-based gameplay rather than pure randomness.

Key Achievements

1. **Complete Functionality:** All 10 functional requirements have been implemented and tested
2. **Probability System:** Both batting styles use proper probability distributions that sum to exactly 1.0
3. **Accurate Power Bar:** Segments are proportionally sized and outcomes are determined by slider position
4. **Smooth Animations:** All required animations (bowling, batting, ball movement) work correctly and are synchronized
5. **Clean Code:** Modular component structure with clear separation of concerns
6. **Bonus Features:** Dynamic commentary system with multiple statements per outcome
7. **Responsive Design:** Application works on both desktop and mobile devices

Technical Highlights

- Proper use of React hooks for state management
- Efficient animation using requestAnimationFrame
- Well-organized component architecture
- Comprehensive code comments
- Meaningful Git commit history (16+ commits)

Learning Outcomes

Through this project, I have gained practical experience in:

- Building interactive web applications with React
- Implementing probability-based game logic
- Creating smooth CSS animations
- Managing complex application state
- Structuring modular, maintainable code
- Using Git for version control with meaningful commits

The application is fully functional, well-documented, and ready for demonstration. All code is original and meets the assignment requirements for CS-4032 Web Programming.

GitHub Repository

URL: https://github.com/SalmanAh/Cricket-Web.git

Total Commits: 16+ meaningful commits showing development progress

---

End of Report

Student Signature: _____

Date: _____